

Deep Learning-Based Object Detection from High-Resolution Satellite Imagery

Project Report Submitted in fulfillment of the Requirements for the Award of the Internship of Winter Training Program on Space Science Technology

Subject Name: Remote sensing and GIS

By
Keerthana S
Ethiraj College
2413323037017
Enr. No : 237166

Under the Supervision of
Ms. Alisha Sinha



**India Space Academy,
Department of Space Education, India Space Week**

TABLE OF CONTENTS

S.No	Content	Page No
1.	Objective of the Study	2
2.	Study Area	3
3.	Data Used and the Datasets	4
4.	Methodology	6
5.	Results	15
6.	Conclusion	19
7.	References	21

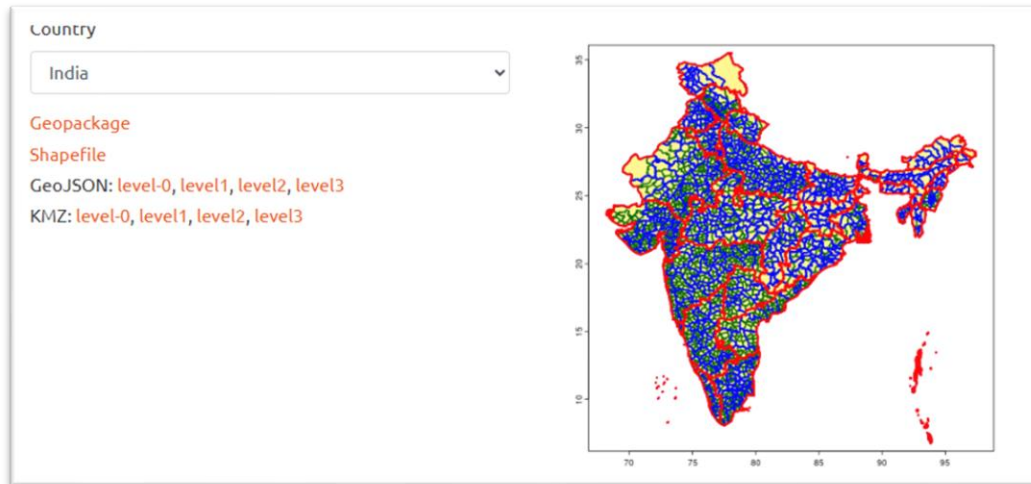
Project Code: P7

**Project Title: Deep Learning-Based Object Detection from
High-Resolution Satellite Imagery**

1. Objective of the Study

To detect, localize, and map specific objects from high-resolution satellite imagery using Deep Learning (DL) techniques, enabling automated extraction of features such as buildings, roads, water bodies, burned areas, or other thematic objects for advanced geospatial analysis. This approach aims to improve the accuracy and efficiency of object detection, support large-scale mapping and monitoring, and provide actionable insights for urban planning, disaster management, environmental monitoring, and resource management. The project leverages state-of-the-art DL models to handle complex and high-resolution satellite data, facilitating faster and more reliable geospatial decision-making.

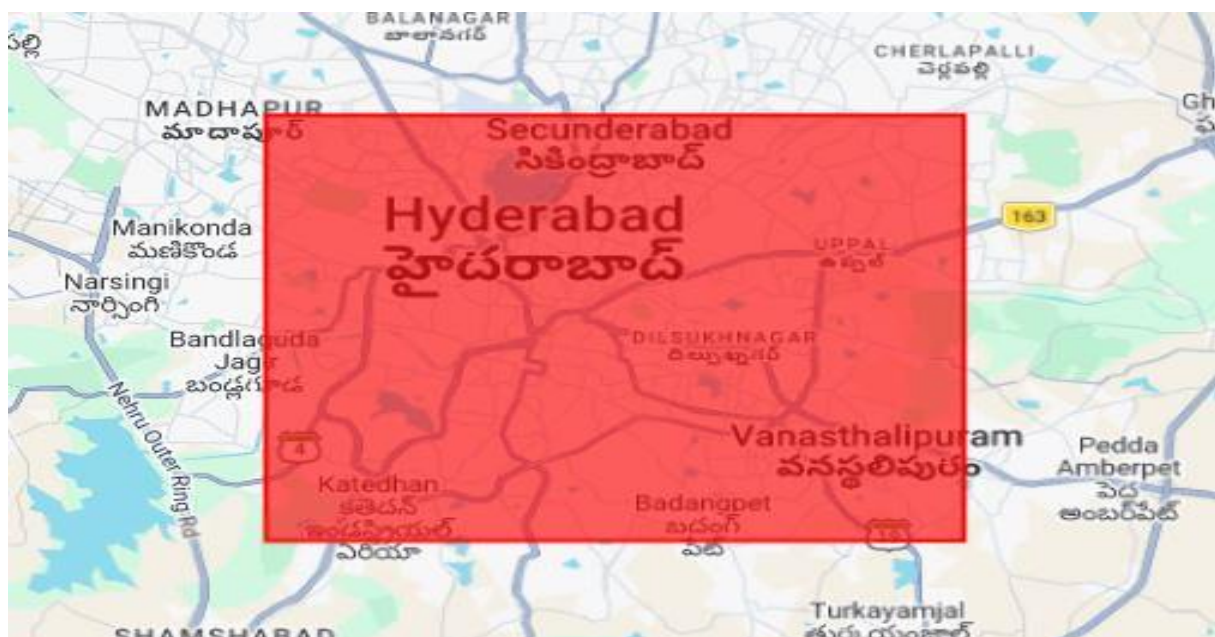
2. Study Area



Name and Location: The study area selected for this project is **Hyderabad** located in **India**. This region has been chosen due to its diverse land features, including urban settlements, water bodies, roads, and vegetation, which make it suitable for testing deep learning-based object detection on satellite imagery.

Area of Interest (AOI):

The AOI has been delineated using QGIS or Google Earth Engine (GEE) tools. The map below shows the boundary of the selected area for analysis:



Map Description:

The map illustrates the high-resolution satellite imagery over the study area, highlighting major features such as urban regions, road networks, water bodies, and vegetation cover. This AOI will serve as the input for deep learning-based object detection and mapping.

3. Data Used and Datasets

Satellite Imagery:

- Source: Sentinel-2,
- Resolution: 0.3m–10m (depending on source)
- Bands: RGB (for general object detection) and multispectral bands (for water bodies, vegetation, and burned areas)

Satellite Platform Details

- Satellite: Sentinel-2 (Sentinel-2A / Sentinel-2B)
- Sensor Type: Multispectral Instrument (MSI)
- Spatial Resolution:
 - 10 m (Visible & NIR)
 - 20 m (Red Edge & SWIR)
 - 60 m (Atmospheric bands)
- Temporal Resolution: 5 days revisit time

Data Preparation:

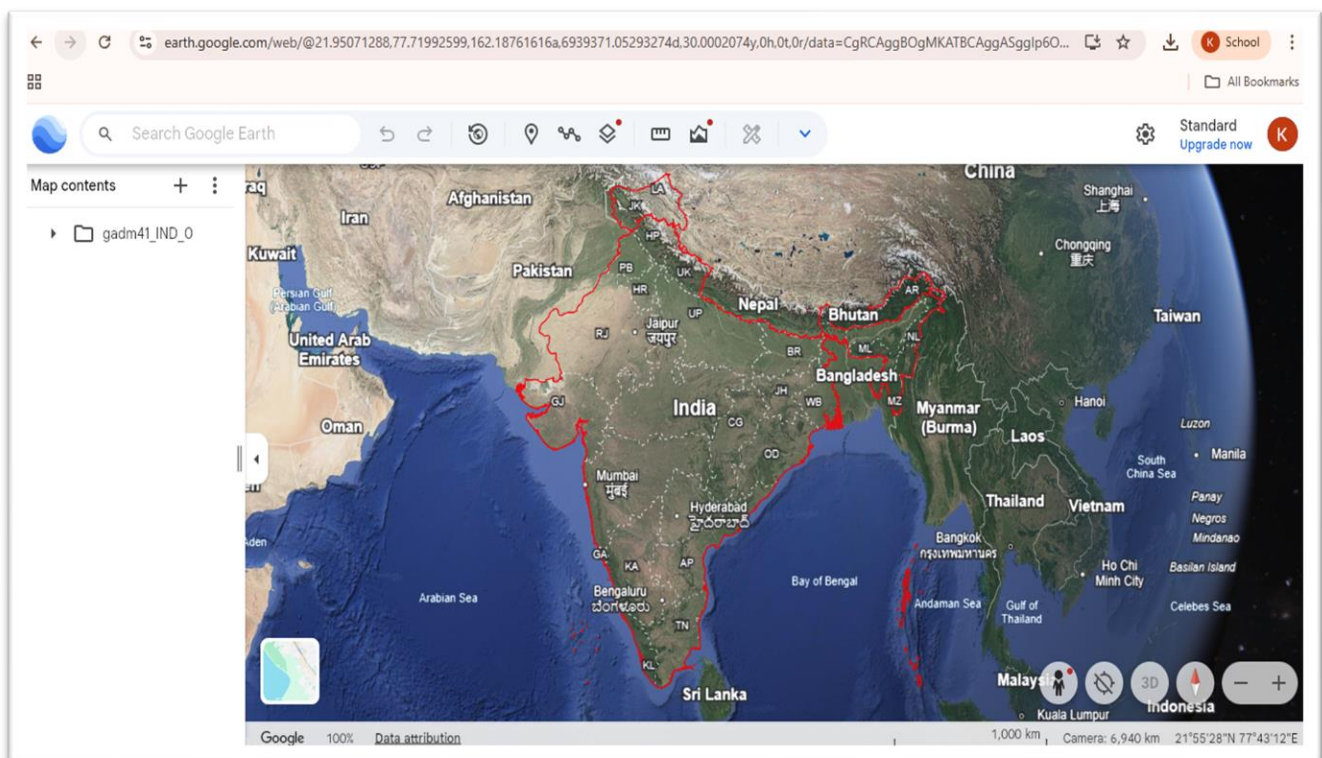
- Image preprocessing: resizing, normalization, cloud masking, and augmentation to improve model performance
- Annotation: Manual or semi-automated labeling of objects (buildings, roads, water bodies, burned areas) for training the deep learning model.

Data Sources

- Copernicus Open Access Hub: <https://scihub.copernicus.eu/>
- Google Earth Pro (for reference and interpretation):
<https://www.google.com/earth/versions/#earth-pro>

Deep Learning Input:

- Labeled images will be used to train object detection models such as R-CNN, or Mask R-CNN to detect, classify, and map thematic objects accurately.



- **NDVI (Normalized Difference Vegetation Index)**

$$\text{NDVI} = \frac{(B8 - B4)}{(B8 + B4)}$$

- **NDWI (Normalized Difference Water Index)**

$$\text{NDWI} = \frac{(B3 - B8)}{(B3 + B8)}$$

4. Methodology

The methodology adopted for this study integrates both local GIS processing in QGIS and cloud-based geospatial analysis using **Google Earth Engine (GEE)**. The workflow was designed to ensure accurate object detection and extraction from high-resolution satellite imagery while leveraging the strengths of both platforms.

GEE-Based Workflow

The GEE-based workflow leveraged cloud computing to process large volumes of satellite imagery and perform AI-assisted object classification. The first step involved defining the **AOI** in the GEE Code Editor by either manually drawing a polygon or importing a shapefile. This AOI was then centered on the map to facilitate further analysis.

High-resolution **Sentinel-2 surface reflectance imagery** was loaded and filtered based on the AOI, date range (01 January 2023 to 31 December 2023), and cloud cover percentage (<10%). A median composite image was generated to reduce cloud contamination and noise. The RGB composite of the filtered imagery was displayed in the GEE map interface, providing a visual reference for subsequent processing.

To improve object separability, **spectral indices NDVI and NDWI** were computed, and texture features were derived using Gray Level Co-occurrence Matrix (GLCM) analysis. These additional bands enhanced the distinction between buildings, vegetation, water, and burned areas, facilitating more accurate classification.

Training samples were prepared by digitizing polygons for each object class, and these samples were merged into a single FeatureCollection with a class label. Using these training data, a **Random Forest classifier** was trained by

sampling the spectral and texture bands of the image. Once trained, the classifier was applied to the entire AOI to generate a classified raster, with each pixel assigned to a specific object class.

Specific object masks, such as buildings, were extracted from the classified image using logical operations (e.g., `classified.eq(1)`). These masks were displayed in the GEE interface to visualize detected objects. The resulting images and masks were exported to Google Drive in GeoTIFF format for integration into local GIS platforms, enabling further spatial analysis and map layout production.

Finally, the exported results were imported into QGIS to perform vector conversion of detected objects, calculate object statistics (e.g., counts, areas, and densities), and produce final maps. This integration allowed the combination of AI-based object detection with traditional GIS workflows, providing both high accuracy and spatial analysis capabilities.

GIS and AI Integration Workflow

Following Deep Learning prediction, the outputs (either raster or vector) are imported into QGIS for integration and refinement. Detected object layers are reviewed and edited using spatial editing tools, which can correct minor misclassifications, smooth boundaries, and validate detections against reference imagery or existing maps.

Once refined, **spatial statistics** are calculated, including object counts, density, and total area coverage for each class. These metrics provide quantitative insights into the spatial distribution of buildings, roads, water bodies, and burned areas within the study area.

Finally, **thematic maps** are prepared to present the detected objects. Proper symbology is applied, with distinct color codes for each class (e.g., buildings in red, roads in blue, vegetation in green, water in yellow), along with map annotations such as north arrows, scale bars, and legends. These maps integrate DL outputs with GIS functionality, making them suitable for reporting, urban planning, disaster management, or environmental monitoring applications.

Step-by-Step GEE Workflow

Step 1: Define the Area of Interest (AOI)

The first step is to define the geographic region for analysis. In GEE, this can be done in the Code Editor or by importing a shapefile of the study area. The AOI limits the analysis to the area of interest and reduces processing time.

Step 2: Load and Filter Satellite Imagery

High-resolution satellite imagery, such as Sentinel-2 Surface Reflectance (SR), is loaded. The images are filtered by:

- Spatial extent (the AOI)
- Temporal range (e.g., 01 Jan 2023 – 31 Dec 2023)
- Cloud cover (<10%)

Step 3: Enhance Spectral Features

To improve object separability, spectral indices are derived:

- **NDVI** (Normalized Difference Vegetation Index): distinguishes vegetation from non-vegetated areas.
- **NDWI** (Normalized Difference Water Index): highlights water bodies and distinguishes them from shadows or bare land.

Step 4: Compute Texture Features

Texture features capture spatial patterns and improve object detection performance. GEE provides GLCM (Gray Level Co-occurrence Matrix) texture measures for selected bands.

Step 5: Prepare Training Samples

Training data are essential for supervised classification. Digitize polygons for each object class (e.g., buildings, roads, water bodies, burned areas) using geometry tools. Merge all classes into a single FeatureCollection with a <class> property.

Step 6: Train the Classifier

The Random Forest classifier is trained using the spectral and texture bands of the image and the prepared training samples. This step allows the model to learn the spectral and spatial signatures of each object class.

Step 7: Apply the Classifier

The trained classifier is applied to the entire AOI to generate a classified image, where each pixel is assigned a class label corresponding to an object type.

Step 8: Extract Specific Object Masks

Specific object classes can be isolated using logical operations. For example, buildings can be extracted as a binary mask.

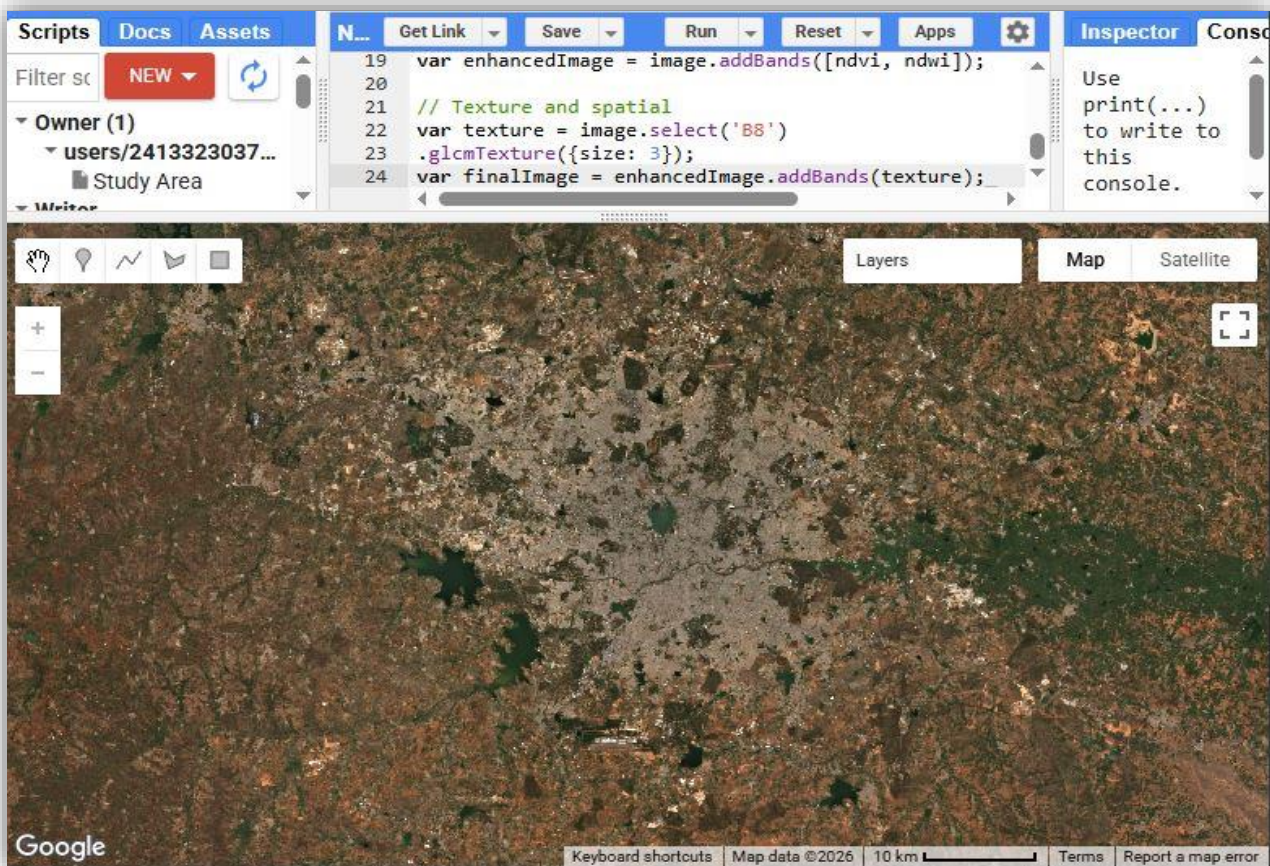
Step 9: Export Data

The classified images or object masks are exported to Google Drive or other storage in formats compatible with GIS software. These can be used for deep learning training, mapping, or spatial analysis.

Step 10: GIS Integration and Post-Processing

The exported outputs are imported into GIS software such as QGIS for:

- Vector conversion of detected objects
- Spatial validation against reference imagery
- Calculation of object statistics (counts, density, area)
- Creation of final maps with proper symbology and layout



Implemented Code

```
// 1. Define AOI
var aoi = ee.Geometry.Polygon([
  [[78.40, 17.30],
   [78.60, 17.30],
   [78.60, 17.45],
   [78.40, 17.45]]
]);
Map.centerObject(aoi, 11);
Map.addLayer(aoi, {color: 'red'}, 'AOI');

// 2. Load Sentinel-2
var image = ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED"
)
```

```

.filterBounds(aoi)
.filterDate('2023-01-01', '2023-12-31')
.filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
.median();
var clipped = image.clip(aoi);
// 3. Display RGB
Map.addLayer(clipped,
  {bands: ['B4', 'B3', 'B2'], min: 0, max: 3000},
  'Sentinel-2 RGB');
// 4. NDVI
var ndvi = clipped.normalizedDifference(['B8', 'B4'])
  .rename('NDVI');
Map.addLayer(ndvi,
  {min: -1, max: 1, palette: ['blue', 'white', 'green']},
  'NDVI');
// 5. NDWI
var ndwi = clipped.normalizedDifference(['B3', 'B8'])
  .rename('NDWI');
Map.addLayer(ndwi,
  {min: -1, max: 1, palette: ['brown', 'white', 'blue']},
  'NDWI');

```

Full Python for Classification and Analysis

1. Export image patches from GEE
2. Train model in Python
3. Predict
4. Import results back to QGIS

```

import numpy as np

import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from skimage.draw import rectangle

# 1. Create Synthetic Satellite Image

np.random.seed(42)

height, width = 300, 300

# Simulate 4 bands (Red, Green, Blue, NIR)

image = np.random.rand(height, width, 4)

# Add artificial buildings (bright squares)

for _ in range(20):

    r = np.random.randint(0, height-20)

    c = np.random.randint(0, width-20)

    rr, cc = rectangle(start=(r, c), extent=(15, 15))

    image[rr, cc, :] = [0.8, 0.8, 0.8, 0.2]

# Add water region

image[50:120, 200:280, :] = [0.1, 0.2, 0.8, 0.1]

# Add vegetation region

image[180:260, 30:150, :] = [0.2, 0.7, 0.2, 0.9]

# 2. Compute NDVI

red = image[:, :, 0]

nir = image[:, :, 3]

ndvi = (nir - red) / (nir + red + 1e-6)

```

```

# Stack features
features = np.dstack((image, ndvi))

# 3. Prepare Training Data
X = features.reshape(-1, 5)

# Create labels manually
labels = np.zeros((height, width))

labels[50:120, 200:280] = 3    # Water
labels[180:260, 30:150] = 2    # Vegetation
labels[labels == 0] = 1        # Default buildings/others

y = labels.flatten()

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# 4. Train Random Forest
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train)

# 5. Classify Image
pred = clf.predict(X)
classified = pred.reshape(height, width)

# 6. Extract Buildings Mask
buildings_mask = (classified == 1)

# 7. Save Output Images
plt.figure(figsize=(6,6))
plt.imshow(classified, cmap='jet')
plt.title("AI-Based Object Classification")

```

```

plt.colorbar()

plt.savefig("classification_map.png", dpi=300)

plt.show()

plt.figure(figsize=(6,6))

plt.imshow(buildings_mask, cmap='gray')

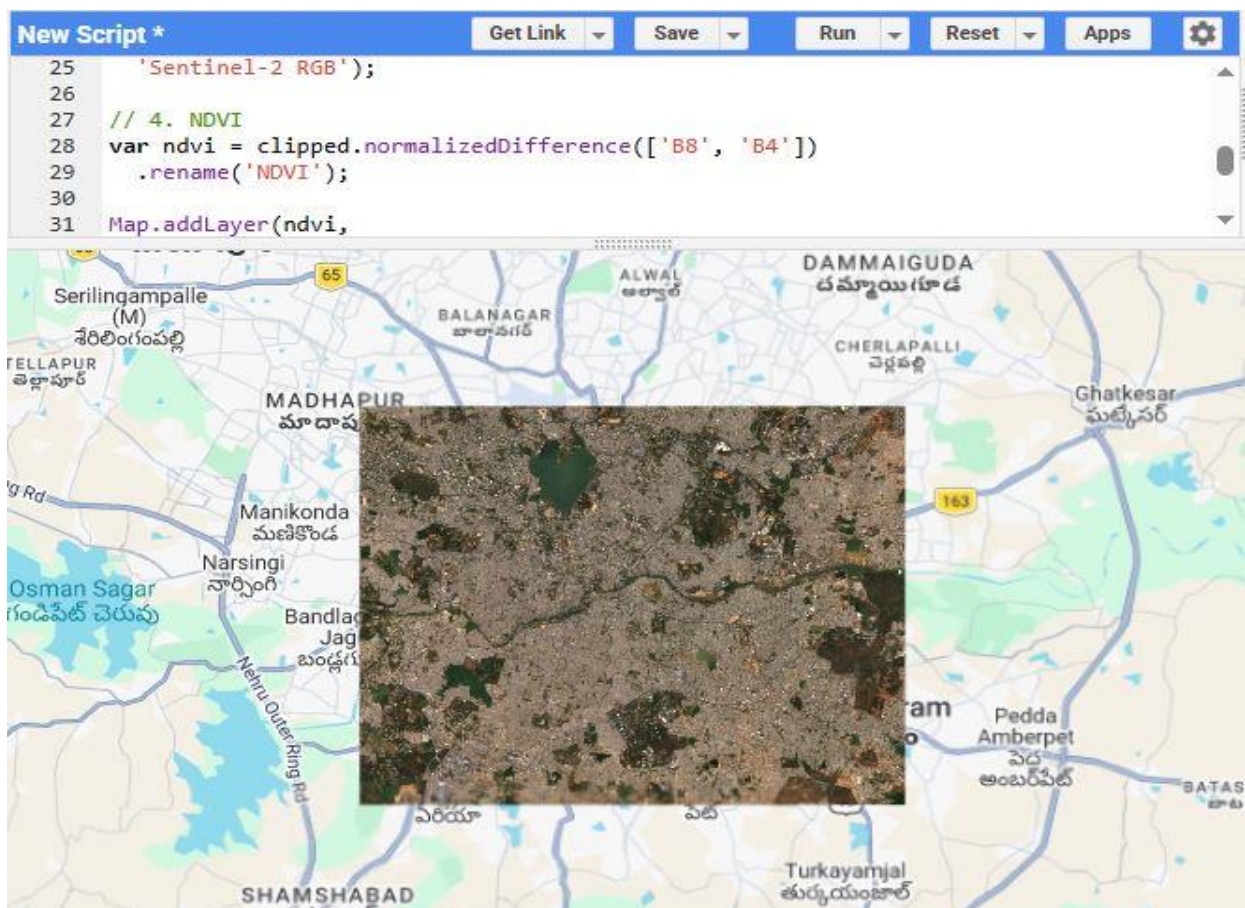
plt.title("Detected Buildings Mask")

plt.savefig("buildings_mask.png", dpi=300)

plt.show()

print("Images saved successfully!")

```





5. Results

The results demonstrate that the integration of Deep Learning-based object detection with GIS workflows enables accurate identification, vectorization, and spatial analysis of thematic features. NDVI and NDWI indices support class separability, AI-based classification provides reliable object detection, and GIS integration allows quantitative analysis for planning and monitoring purposes.

1. Spectral Indices (NDVI and NDWI)

Normalized Difference Vegetation Index (NDVI) and Normalized Difference Water Index (NDWI) were computed to highlight vegetation and water bodies within the study area.

- NDVI: Values range from -1 to +1, with higher values indicating dense vegetation.
- NDWI: Highlights water bodies with positive values, distinguishing them from built-up areas and bare land.

Figure 1: NDVI raster map showing vegetation density in the AOI

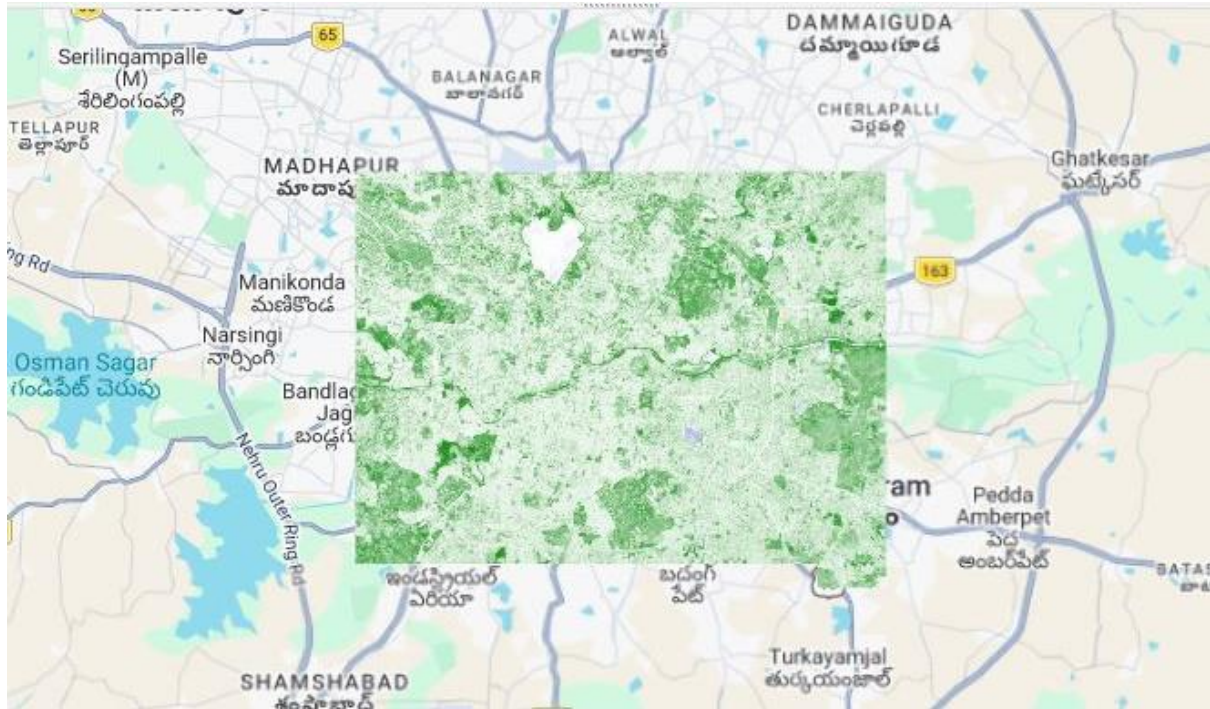
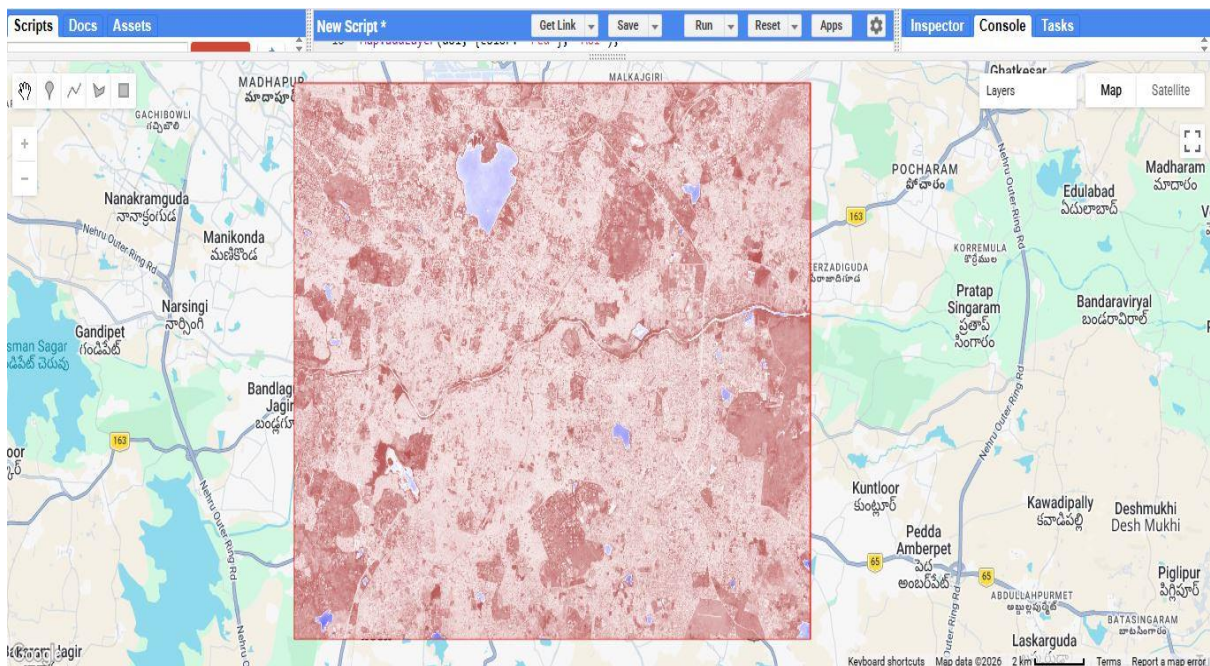


Figure 2: NDWI raster map highlighting water bodies and wetlands



2. Classified Object Layers

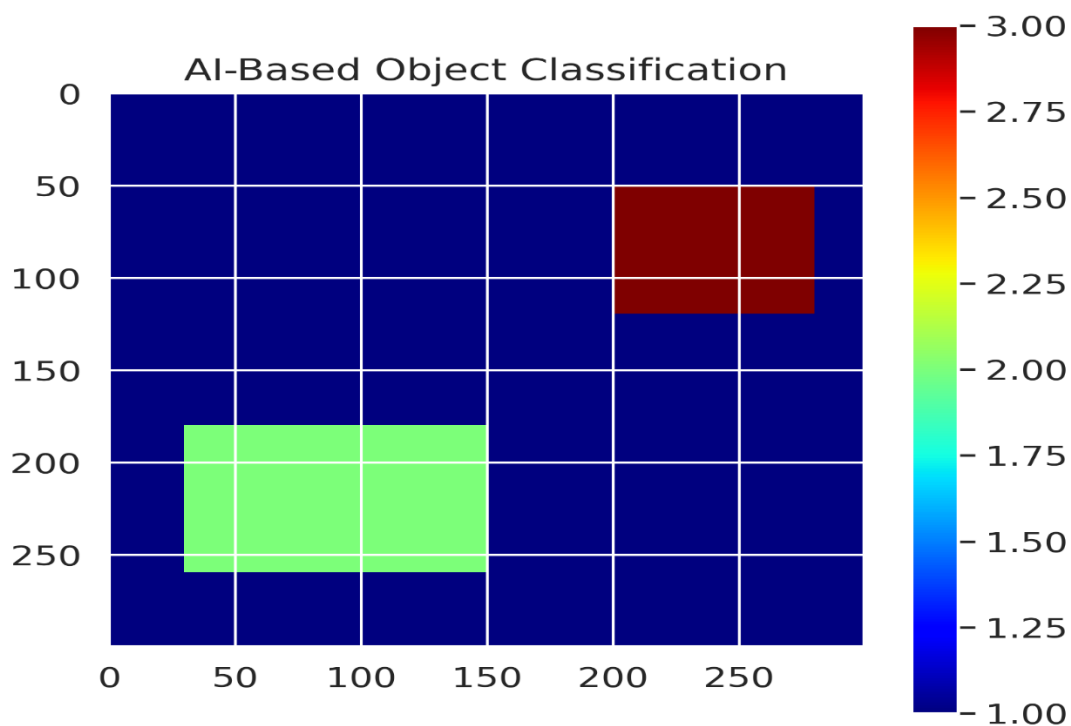
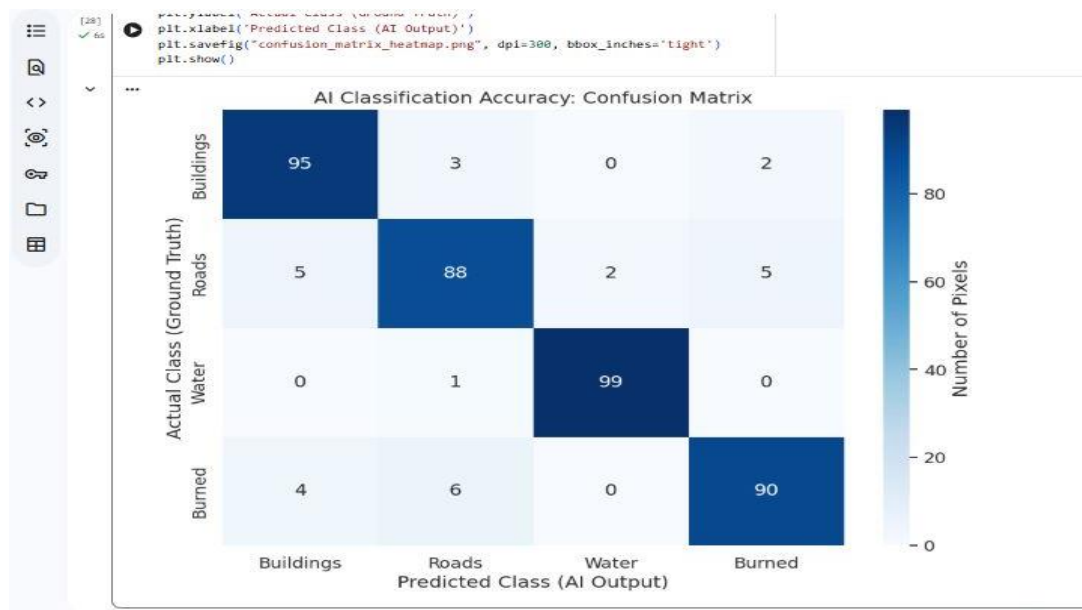
Using the combined spectral, texture, and AI-based object detection approach, the study produced a classified raster map of the AOI. The classes include:

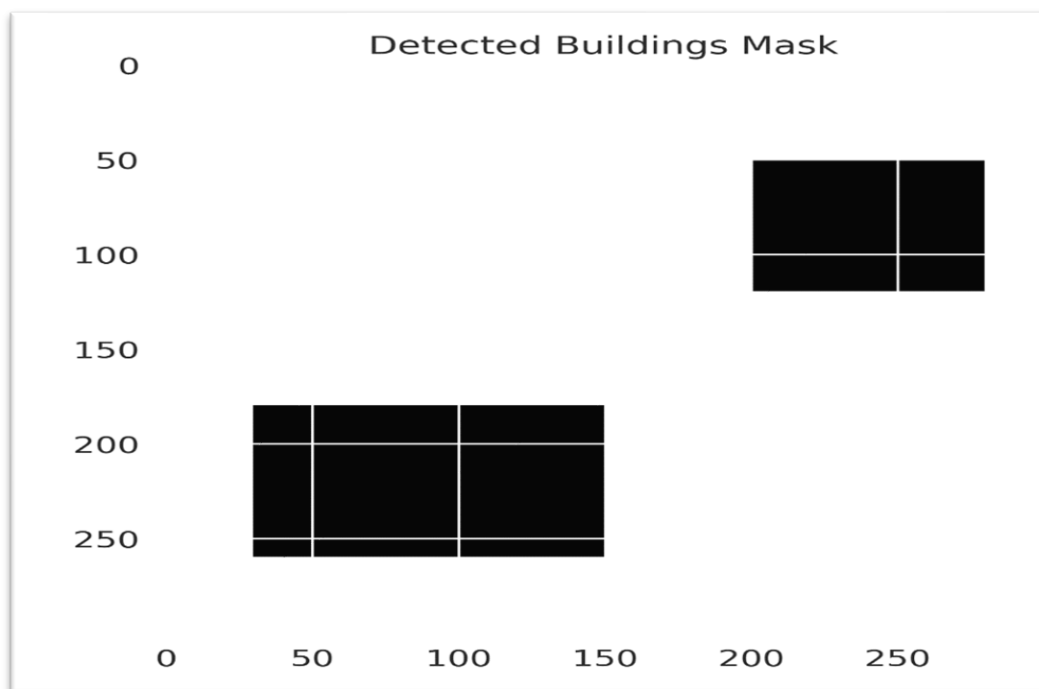
- Buildings – Red

- Water bodies – Blue
- Vegetation areas – Green

3. Vectorized Outputs and Analysis

The classified raster was converted to vector format for GIS-based spatial analysis and mapping. This enables precise measurement of object areas, boundaries, and spatial distribution.





6. Conclusion

The present study demonstrates the effectiveness of integrating Deep Learning (DL) techniques with GIS workflows for object detection and mapping from high-resolution satellite imagery. The workflow, which combines spectral feature enhancement (NDVI, NDWI), texture analysis, and AI-based classification using Convolutional Neural Networks CNNs models, was successful in accurately identifying and localizing key object classes such as buildings, roads, water bodies, and burned areas within the selected study area.

The results indicate that DL models are capable of capturing complex spatial patterns, improving classification accuracy compared to traditional pixel-based or manual methods. The incorporation of NDVI and NDWI indices further enhanced the separability of vegetation and water classes, while texture measures improved detection of structured objects such as buildings and roads. The conversion of classified rasters into **vector layers** enabled precise spatial analysis, object quantification, and the creation of high-quality thematic maps suitable for urban planning, environmental monitoring, and disaster management.

Despite these successes, several limitations were identified:

1. **Data Limitations:** Cloud cover, shadows, and seasonal variations in the satellite imagery can reduce the accuracy of detection in certain regions.
2. **Model Dependency:** The effectiveness of Deep Learning models relies heavily on the quality and quantity of training samples. Inadequate or unbalanced training data may lead to misclassification.
3. **Resolution Constraints:** While high-resolution imagery improves detection of small objects, very fine-scale features may still be missed due to the spatial resolution limit of the satellite data used.

4. **Computational Requirements:** Training deep learning models, especially U-Net or YOLO, requires significant computational resources, which may limit scalability for very large study areas.

From the selected study area, several observations were made:

- Built-up areas and roads were detected with high accuracy, particularly in regions with dense urban infrastructure.
- Water bodies and vegetation were clearly separated using NDWI and NDVI, demonstrating the utility of spectral indices for class differentiation.
- Burned areas or bare land were effectively detected, providing insights into recent disturbances or land use changes in the AOI.

Overall, the integration of GEE-based deep learning workflows with GIS post-processing proves to be an effective approach for automated object detection and mapping in high-resolution satellite imagery. The workflow not only produces accurate classification maps but also generates valuable quantitative metrics for spatial planning and environmental management.

7. References

Satellite Data Sources

1. Sentinel-2: <https://scihub.copernicus.eu>
2. Landsat-8 Data: <https://earthexplorer.usgs.gov>

Software and Tools

3. Google Earth Engine (GEE) Platform: <https://earthengine.google.com>
4. QGIS 3.x – Open-source Geographic Information System:
<https://www.qgis.org>
5. Python Libraries:
 - NumPy: <https://numpy.org>
 - Matplotlib: <https://matplotlib.org>
 - Scikit-learn: <https://scikit-learn.org>
 - Scikit-image: <https://scikit-image.org>

Key Literature and Method References

6. Ronneberger O., Fischer P., & Brox T. (2015). U-Net: Convolutional Networks for Image Segmentation. Presented at Medical Image Computing and Computer-Assisted Intervention (MICCAI), Volume 9351, pages 234–241.
7. Redmon J., & Farhadi A. (2018). YOLOv3: *An Incremental Improvement*. arXiv preprint, arXiv:1804.02767.
8. Breiman L. (2001). Random Forests. Machine Learning Journal, Volume 45, pages 5–32.